



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

PaN-Finder Deliverable 3.b:

Report on benchmark and test pairs

Table of Contents

1 Introduction.....	3
2 Benchmarking Methodologies.....	5
2.1 Development Benchmarks.....	5
2.2 Evaluation Run Benchmarks.....	5
2.3 Evaluation Test Pair Benchmarks.....	6
3 Implementation Details.....	7
3.1 Execution Workflow.....	7
3.1.1 Development Benchmarks Workflow.....	7
3.1.2 Evaluation Benchmarks Workflow.....	7
3.2 Hyperparameter Configuration Files.....	8
3.3 Selected RRF Configuration Patterns.....	9
3.4 Scoring Algorithms.....	10
3.4.1 Development Benchmark Scoring.....	10
3.4.2 Evaluation Benchmarks Scoring.....	11
3.5 Database Schema.....	12
3.6 Metric Definitions.....	13
4 Results and Analysis.....	15
4.1 Development Benchmark Results.....	15
4.1.1 Output Files Generated.....	15
4.1.2 Score Interpretation.....	15
4.1.3 Examples.....	15
4.2 Evaluation Benchmark Runs Results.....	16
4.2.1 Statistical Summary.....	16
4.2.1.1 Comprehensive (All) Test Pairs.....	16
4.2.1.2 Synthetic Test Pairs.....	16
4.2.1.3 Expert Test Pairs.....	16
4.2.2 Performance Differences Analysis.....	17
4.2.2.1 Comprehensive vs Synthetic.....	17



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

4.2.2.2 Expert Test Pairs Performance Gap.....	17
4.2.2.3 Relative Performance Order.....	17
4.2.3 Temporal Coverage.....	18
4.3 Evaluation Benchmark Test Pair Results.....	18
4.3.1 Statistic Summary.....	18
4.3.2 Statistic by RRF configuration.....	19
4.3.3 Temporal Pattern.....	20
5 Visualization.....	21
5.1 Development Benchmark.....	21
5.2 Evaluation Benchmark Plots.....	23
5.2.1 Evaluation Run Metrics.....	23
5.2.1.1 Mean Context Precision (MCP).....	23
5.2.1.2 Mean Presence (Mps).....	24
5.2.1.3 Mean Linear Zero Position (Mlzp).....	24
5.2.1.4 Mean Exponential Zero Position (Mezp).....	25
5.2.2 Evaluation Test Pairs Metrics.....	26
5.2.2.1 Metrics Scores over time.....	26
5.2.2.2 Metrics Grouped by RFF Configuration.....	27
5.2.2.3 Metrics Distribution.....	28
6 Strengths and Limitations.....	31
6.1 Strengths.....	31
6.2 Limitations.....	32
7 Conclusion and Recommendations.....	34
7.1 Summary of Findings.....	34
7.1.1 Performance Overview.....	34
7.1.2 Temporal Stability.....	34
7.1.3 RRF Configuration Insights.....	34
7.2 Recommendations.....	35
7.2.1 For Production Deployment.....	35
7.2.2 For Testing Infrastructure.....	35
7.2.3 For Future Development.....	36
8 Acknowledgements.....	37
9 Funding.....	37



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

1 Introduction

The PaN-Finder benchmarking system provides a comprehensive framework for evaluating the performance of the search functionality across multiple configurations and test scenarios.

The PaN-Finder project consolidates findings from three complementary benchmarking approaches:

1. **Development Benchmarks:**

Runs a single presence metric on predefined query datasets using the `multi_config_evaluator.py` script to test predefined RRF (Reciprocal Rank Fusion) configurations quickly during integration testing.

2. **Evaluation Benchmarks:**

Uses the `hyperparameters_runner.py` to a selected hyperparameters configuration that selects all the configurable parameters and variables of the system including which test pair sets. Most of the time, all the test pairs are used, synthetic and experts. Each evaluation benchmarks produces two types of metrics:

1. **Run Metrics:**

These are average metrics across all the test pairs.

Usually for each run, the same metric is computed by averaging across all the test pairs (Comprehensive), the synthetic test pairs and the experts ones, providing 3 different averages.

2. **Test Pair Metrics:**

These metrics are computed for each test pair, allowing to evaluate the effects of the changes introduced for a specific test pair and RRF configuration.

By combining automated testing, temporal tracking, and statistical analysis, the benchmarks ensure a robust evaluation of different ranking strategies and configurations, and provide a multi-faceted view of the PaN-Finder's search performance and accuracy.

The benchmarking infrastructure supports:

- Automated execution of search queries across multiple system's configurations
- Comprehensive metric collection. Multiple metrics are defined and collected, although in this moment in time, only the ones listed below are actively used. Please refer to deliverable **2.b** and **3.c** for more information on the metrics.
 - *Context Precision (CP)*,
 - *Presence (Ps)*,
 - *Linear Zero Position (lzp)*,



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- *Exponential Zero Position (ezp)*
- Storage of all results in a PostgreSQL database for historical tracking and analysis
- Visualization of performance trends over time and across configurations



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

2 Benchmarking Methodologies

2.1 Development Benchmarks

The development benchmarks evaluate the PaN-Finder application using synthetic query datasets and predefined test configurations. This approach uses the `run_multi_config_evaluator.sh` script to orchestrate benchmark execution against the `multi_config_evaluator.py` Python script.

Key Characteristics:

- Tests 4 predefined RRF configurations:
 - MatchFocus,
 - SimilarityFocus,
 - 6_6_3_3_10,
 - LessKeyword
- Uses synthetic query datasets from `benchmark/queries/synthetic/`
- Implements linear decay scoring: $\text{score} = \max(0.0, 1.0 - (\text{offset} / 20.0))$
- Tracks performance over time with overall changes plots
- Generates CSV and JSON output files for detailed analysis

2.2 Evaluation Run Benchmarks

The Evaluation Run Benchmarks expand the testing scope through the `run_hyperparameters_runner.sh` script, which automatically processes the selected JSON hyperparameters configuration file stored in the `hyperparameters/` directory. This approach allows testing the application towards a specific configuration.

Key Characteristics:

- Tests a hyperparameters file from the `hyperparameters/` directory
- Uses PostgreSQL database (`pan-finder-benchmarks`) for result storage
- Supports all types of test pairs
- Computes all defined metrics, although only metrics CP, Ps, lzp, and ezp are considered at the moment
- Currently stores more than 2000 total metric measurements across almost 100 completed runs
- At the moment, spans from August 5, 2025, to present with regular testing



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

2.3 Evaluation Test Pair Benchmarks

The Evaluation Test Pair Benchmarks provide the most detailed analysis, focusing on each individual test pair with multiple runs with all metrics across 5 unique RRF configurations. This analysis examines performance by configuration type and individual run.

Key Characteristics:

- Analyzes 5 RRF configurations:
 - Similarity Focus,
 - Match Focus,
 - 6_6_3_3_10,
 - Standard,
 - Less Keyword
- Tests with all type of test pairs: positive, extended presence, and comparative relative presence.
- Leverages the knee point algorithm to organize results set in groups from exact match to suggested
- Stores all results in PostgreSQL database



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

3 Implementation Details

3.1 Execution Workflow

3.1.1 Development Benchmarks Workflow

The `run_multi_config_evaluator.sh` script performs the following steps:

1. Validates presence of `.env` file containing configuration
2. Creates a Python virtual environment if one doesn't exist
3. Installs dependencies from `requirements.txt`
4. Activates the virtual environment
5. Executes `src/multi_config_evaluator.py`

The Python script implements:

1. **Configuration Loading:**
Loads RRF k-values from `rrf_score_k_values_matrix.json` and synthetic query datasets
2. **Query Processing:**
Uses LLM (currently GPT-4.1-mini) to extract search parameters, constructs SQL queries with `SearchQueryBuilder` function.
3. **Scoring Algorithm:**
Identifies target DOI position, calculates linear decay score
4. **Aggregation:**
Computes average scores per dataset, overall averages and run times

3.1.2 Evaluation Benchmarks Workflow

The `run_hyperparameters_runner.sh` script:

1. Checks for `.env` file and loads environment variables
2. Sets up Python virtual environment with dependencies
3. Load the selected JSON file from the `hyperparameters/` directory
4. Execute benchmark using `src/hyperparameters_runner.py` with the `-p` flag

The `hyperparameters_runner.py` script:

1. Registers connection to PostgreSQL database
2. Loads system prompt based on hyperparameters configuration



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

3. Initializes `SearchQueryBuilder` instance with sentence transformer and RRF score weights
4. Creates unique run ID and records run in database
5. Retrieves test pairs from database based on configuration
6. Processes each test pair:
 1. extracts components,
 2. builds query,
 3. executes query,
 4. computes metrics
 5. store metrics' values in database
7. Stores run-level metrics and updates run with completion timestamp.

3.2 Hyperparameter Configuration Files

Each hyperparameters JSON file defines a specific benchmark configuration with the following structure:

1. **Metadata:**
 - *id*: Unique UUID identifier
 - *name*: Human-readable name (e.g., “Similarity Focus 20 v6”, “Standard k 6 20 v4”)
 - *notes*: Descriptive notes about the configuration strategy
2. **Application Configuration:**
 - *system_prompt*: LLM model and prompt version
 - *embeddings*: Sentence transformer model (e.g., “all-MiniLM-L12-v2”)
 - *openai*: OpenAI API configuration
 - *temperature*: Typically 0 for deterministic results
 - *max_tokens*: Maximum tokens for LLM responses
 - *seed*: Random seed for reproducibility
3. **Score Weights**: RRF k-values for different score types.
Note: Lower k-values give more weight to top results; higher k-values distribute weight more evenly
 - *rrf_k_similarity*: K-value for similarity scores
 - *rrf_k_chunk*: K-value for chunk similarity scores
 - *rrf_k_full_match*: K-value for full match scores
 - *rrf_k_partial_match*: K-value for partial match scores
 - *rrf_k_keyword*: K-value for keyword scores
4. **Results Set:**



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- *results_set_size*: Number of results to return (typically 10 or 20)

5. Test Pairs:

- *test_pairs_set*: Which test pairs set to use: comprehensive (all), synthetic, or expert.
- *test_pairs_type*: Type of test pairs: all, positive, etc. Currently used

6. Metrics Configuration:

- *test_pair_metrics*: List which metrics needs to be compute for each test pair type
- *run_metrics*: List of Run-level aggregated metrics. Each aggregated metrics is computed for all test pairs, and also for synthetic and expert ones

3.3 Selected RRF Configuration Patterns

The following list provides information about the RRF weights configuration that have been selected and used in the PaN-Finder benchmarks, so far. The RRF weights are the weights used to compute the overall score from each individual score assigned by the component queries to each individual result. The overall score is then used to rank the results and select the top n ones that are presented to the user.

Following is the list of the RRF weight configurations selected and used in the benchmarks:

- **Pattern Name:** Standard
Description: this RRF configuration gives equal important to all components. Multiple sources suggest to use 6 as the standard weight.
RRF k weights:
 - *Similarity*: 6
 - *Chunk*: 6
 - *Full Match*: 6
 - *Partial Match*: 6
 - *Keywords*: 6
- **Pattern Name:** Similarity Focus
Description: this RRF configuration favor similarity match on summary and chunks over the other components, while reduced the importance of the keywords one.
RRF k weights:
 - *Similarity*: 1
 - *Chunk*: 1
 - *Full Match*: 6
 - *Partial Match*: 6
 - *Keywords*: 10



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- **Pattern Name:** Match Focus

Description: this RRF configuration gives higher importance to the match components (full and partial), while slightly reducing similarity and chunks and further reduce keywords.

RRF k weights:

- *Similarity:* 8
- *Chunk:* 8
- *Full Match:* 1
- *Partial Match:* 1
- *Keywords:* 10

- **Pattern Name:** Less Keywords

Description: this RRF configuration maintains standard equal weight for all the components, while reducing the importance of the keywords one.

RRF k weights:

- *Similarity:* 6
- *Chunk:* 6
- *Full Match:* 6
- *Partial Match:* 6
- *Keywords:* 10

- **Pattern Name:** 6_6_3_3_10

Description: this RRF configuration maintains standard weight for similarity and chunks, while slightly increase importance for matches and reduce keywords.

RRF k weights:

- *Similarity:* 6
- *Chunk:* 6
- *Full Match:* 3
- *Partial Match:* 3
- *Keywords:* 10

3.4 Scoring Algorithms

3.4.1 Development Benchmark Scoring

For each query, the system perform the following steps:

1. Identifies the target DOI's position in the results



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

2. Calculates offset: $\text{offset} = \max(0, \text{position} - \text{min_position})$
3. Computes score: $\text{score} = \max(0.0, 1.0 - (\text{offset} / 20.0))$

This creates a linear decay score providing a full credit score of 1 if target document is at or before *min_position*, although at the moment *min_position* is always 0. The farther down the list the results is, the closer the score is to 0.

Component scores are:

- Similarity Score,
- Chunk Similarity Score,
- Full Match Score,
- Partial Match Score,
- Keyword Score

3.4.2 Evaluation Benchmarks Scoring

Evaluation benchmarks computes the metrics selected in the hyperparameters file. Given that we are still building the necessary body of knowledge to select which is the most performing metrics, we have decided to compute all of them so far. Although we are still lacking a final decision, we can already see that the ones that are so far meaningful are the following test pair metrics:

- **CP:** Context Precision
- **Ps:** Presence
- **lzp:** Linear Zero Position
- **ezp:** Exponential Zero Position

It is worth mentioning that while we were in the process of writing all the deliverables, we realized that additional reacher metrics were needed to further improve the accuracy of the application. Such metrics have been implemented, but not enough values have been collected yet, so have been excluded from this deliverable. The metrics are:

- **EP:** Extended Presence
- **CRP:** Comparative Relative Presence

We invite the reader to read deliverable 3.c for more information about this metrics, their definition and meaning.

Regarding Run-Level metrics, for each run, the average of all the metrics are computed and saved, in order to evaluate how the configuration tested performs compared to other configurations. The run levels metrics use the abbreviation of the test pair metric with the addition of a prefixes indicating which type of test pair has been computed on. The prefix c (Comprehensive) indicates that all test pairs have been included. Prefix s indicates that only



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

synthetic test pairs have been used, while prefix e indicates that only expert test pairs have been used. Currently the only run level metrics is average or mean value, so the second letter is always M.

3.5 Database Schema

All evaluation benchmark results are stored in a PostgreSQL database (usually named `pan-finder-benchmarks`) with the following schema:

1. **benchmarks_run**: Stores metadata about each benchmark run
 - *runid* (UUID): Unique identifier
 - *startdate* (timestamp): When the run started
 - *enddate* (timestamp): When the run completed
 - *hyperparameters* (JSONB): Full hyperparameters configuration
2. **benchmarks_run_test**: Stores results of each test pair execution
 - *testId* (UUID): Unique test execution ID
 - *runId* (UUID): Reference to the benchmark run
 - *tpId* (UUID): Reference to the test pair
 - *intention, keywords, filters*: Extracted query components
 - *rank*: Rank of target DOI in results
 - *runTime, extractionTime, buildingTime, queryTime*: Timing metrics
 - *sqlQuery*: The generated SQL query
 - *resultsSet*: JSON array of all results with scores
 - *kneePointResults*: JSON with knee point computation results
 - *ranks, actualGroups*: Arrays for comparative tests and metrics
3. **test_pair_metrics_values**: Stores individual metric values for each test pair execution
 - *id* (UUID): Unique identifier
 - *runId* (UUID): Reference to the benchmark run
 - *testId* (UUID): Reference to the test execution
 - *metric* (text): Metric name
 - *value* (numeric): Metric score value
4. **run_metrics_values**: Stores aggregated run-level metrics
 - *id* (UUID): Unique identifier
 - *runId* (UUID): Reference to the benchmark run
 - *metric* (text): Aggregated metric name
 - *value* (numeric): Aggregated metric value
5. **test_pairs**: Reference table containing all test pair definitions



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- *id* (UUID): Unique identifier
- *user_prompt* (string): prompt entered to the by the user ,
- *target_doi* (string): DOI of the target document, aka the document that we are looking for

3.6 Metric Definitions

This section lists the test pair metrics that are used in the evaluation benchmarks. This list is a summary extracted from the full set information contained in deliverable 3.b.

- **Metric:** CP

Full name: Context Precision

Description: this metric evaluates whether the target item is present in the results set with a higher value, the closer the item is to the top position. It is rank aware.

Definition:

$$CP = \begin{cases} \frac{1}{r} & \text{if } 0 < r \leq N \\ 0 & \text{otherwise} \end{cases}$$

where:

- *r* = rank of the relevant item in the results set
- *N* = Number of items in the results set

- **Metric:** Ps

Full name: Presence

Description: this metric indicate if the target item is present in the results set.

Definition:

$$Ps = \begin{cases} 1 & \text{if } 0 < r \leq N \\ 0 & \text{otherwise} \end{cases}$$

where:

- *r* = rank of the relevant item in the results set
- *N* = number of items in the results set



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- **Metric:** lzp

Full name: Linear Zero Position

Description: this metrics assign a score from 1 to 0 that linearly decrease with the rank from the first position to the last one. This metric has rank awareness.

Definition:

$$lzp = \begin{cases} 1 - \frac{r-1}{N} & \text{if } 0 < r \leq N \\ 0 & \text{otherwise} \end{cases}$$

where

- r = rank of the relevant item in the results set
- N = number of items in the results set

- **Metric:** ezp

Full name: Exponential Zero Position

Description: this metrics assign a score from 1 to 0 that exponentially decrease with the rank. First position has a score of 1, it zeros out at $K+1$. This metric has rank awareness.

Definition:

$$ezp = \begin{cases} \frac{e^{\frac{1}{S}(N-r)} - 1}{e^{\frac{N}{S}} - 1} & \text{if } 0 < r \leq N \\ 0 & \text{otherwise} \end{cases}$$

where

- r = rank of the desired document in the results set
- N = number of items in the results set
- S = Curvature Scaling Constant with $S \in \mathbb{Z}_+^*$. Suggested value 1/3.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

4 Results and Analysis

4.1 Development Benchmark Results

4.1.1 Output Files Generated

Every time a development benchmark is run the following files are created:

1. **CSV Results File** (`results_YYYYMMDD_HHMMSS.csv`):
which contains per-dataset average scores and percentages for each test configuration with the following columns:
 - `test_name`,
 - `dataset_name`,
 - `avg_score`,
 - `avg_score_percent`
2. **JSON Raw Scores** (`raw_scores_by_test_config_YYYYMMDD_HHMMSS.json`)
which contains all individual query scores organized by test configuration
3. **Visualizations** which are a set of multiple files:
 - `average_scores_per_dataset_*.png`:
Bar chart showing average scores per dataset
 - `score_distribution_boxplot_*.png`:
Box plot visualizing score distribution
 - `score_distribution_stripplot_*.png`:
Strip plot showing individual score distribution

4.1.2 Score Interpretation

- **Score Range:** 0.0 to 1.0, where 1.0 = perfect (target at position 0 or before `min_position`)
- **Percentage:** Scores multiplied by 100 (e.g., 0.85 = 85%)
- **Position Impact:** Each rank beyond `min_position` reduces score by 5% ($1/20 = 0.05$)
- **Decay Window:** Scores decay over 20 ranks, then hit 0

4.1.3 Examples

Example findings from `results_20251028_115959.csv`:

- Match Focus configuration average score: 74.25%



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- Similarity Focus configuration average score: 70.5%

Over all the test performed, following is a summary of the configurations performance:

- **Less Keyword:** generally performs best
- **6_6_3_3_10:** strong performance
- **Match Focus:** lower average performance despite conceptual appeal for precise matching
- **Similarity Focus:** slightly lower performance than *Less Keyword* and *6_6_3_3_10*

4.2 Evaluation Benchmark Runs Results

4.2.1 Statistical Summary

This section presents some statistics about run metrics.

The run statistics are named after the test pair metric together with the set pair selected. For example: cMPs is composed by three parts: c M and Ps. It stands for Mean of metrics Presence across all test pairs (comprehensive)

4.2.1.1 Comprehensive (All) Test Pairs

cMCP	102	0.7064	0.7071	0.0245	0.6662	0.7521
cMPs	102	0.8448	0.8543	0.0276	0.7649	0.8874
cMezp	102	0.7291	0.7291	0.0235	0.6836	0.7722
cMlzp	102	0.7927	0.7969	0.0265	0.7212	0.8401

4.2.1.2 Synthetic Test Pairs

sMCP	102	0.7268	0.7242	0.0254	0.6807	0.7722
sMPs	102	0.8563	0.8660	0.0270	0.7732	0.8935
sMezp	102	0.7498	0.7472	0.0241	0.6974	0.7917
sMlzp	102	0.8095	0.8174	0.0264	0.7330	0.8500

4.2.1.3 Expert Test Pairs

eMCP	102	0.2255	0.2189	0.0521	0.1206	0.3273
eMPs	102	0.5722	0.5714	0.1238	0.3571	0.7857
eMezp	102	0.2435	0.2437	0.0691	0.1121	0.3824
eMlzp	102	0.4013	0.3786	0.1011	0.2143	0.5909



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

4.2.2 Performance Differences Analysis

4.2.2.1 Comprehensive vs Synthetic

Comprehensive and synthetic test pairs exhibit strong performance with synthetic pairs showing marginally higher mean scores. *sMPs* (average score 0.8563) achieves the highest mean score of all metrics, followed by *sMlzp* (average score 0.8095). Comprehensive test pairs perform nearly as well, with *cMPs* at average score of 0.8448 and *cMlzp* at average score of 0.7927. Low standard deviations (0.0235-0.0276) indicate consistent performance.

4.2.2.2 Expert Test Pairs Performance Gap

Expert test pairs demonstrate substantially lower performance across all metrics. *eMCP* has the lowest mean score at 0.2255, with *eMezp* nearly as low at 0.2435. Higher standard deviations (0.0521-0.1238) for expert metrics indicate greater variability, reflecting increased difficulty and sensitivity to configuration changes. The application clearly needs to be improved when dealing with expert metrics.

4.2.2.3 Relative Performance Order

Across all test pair sets, Presence (Ps) and Linear Zero Position (lzp) metrics consistently outperform Context Precision (CP) and Exponential Zero Position (ezp).



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

4.2.3 Temporal Coverage

Date	Runs Completed	Most Common RRF Configuration
2025-08-05	5	
2025-08-06	1	
2025-08-07	12	6_6_3_3_10
2025-09-09	3	Similarity Focus
2025-09-10	5	Similarity Focus
2025-09-11	10	6_6_3_3_10
2025-09-12	32	Less Keywords
2025-09-13	2	Similarity Focus
2025-10-31	23	Similarity Focus
2025-11-01	5	Similarity Focus
2025-12-08	7	Similarity Focus
2025-12-09	11	Similarity Focus
2025-12-10	11	Similarity Focus
2025-12-15	16	Similarity Focus
2025-12-17	10	Similarity Focus
2025-12-18	32	6_6_3_3_10
2026-07-16	2	Similarity Focus
2026-07-20	9	Similarity Focus

Total: 196 benchmark runs, 98 completed runs, 2829 total metric measurements

4.3 Evaluation Benchmark Test Pair Results

4.3.1 Statistic Summary

At the moment of this writing, we have 117 runs with complete CP, Ps, lzp, ezp metrics across 5 RRF configurations. Following is the table containing the averages for the four metrics

Configuration	# Runs	CP Mean	Ps Mean	lzp Mean	ezp Mean
Similarity Focus	48	0.208	0.627	0.368	0.201
Match Focus	7	0.259	0.571	0.436	0.266
6_6_3_3_10	17	0.237	0.546	0.382	0.257
Less Keywords	21	0.233	0.582	0.431	0.265
Standard	24	0.247	0.589	0.443	0.279



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

4.3.2 Statistic by RRF configuration

In the following section, we are reporting the detailed information about each metrics organized by configuration:

- **Configuration:** Similarity Focus

Runs: 48 (most runs)

Performance: Lowest average CP and ezp, highest Ps variance

Metric	Min	Max	Mean	Std
CP	0.121	0.307	0.208	0.057
Ps	0.357	1.000	0.627	0.192
lzp	0.214	0.564	0.368	0.091
ezp	0.112	0.328	0.201	0.056

- **Configuration:** Match Focus

Runs: 7 (fewest runs)

Performance: Highest average CP, lzp, ezp; most consistent

Metric	Min	Max	Mean	Std
CP	0.170	0.312	0.259	0.057
Ps	0.364	0.727	0.571	0.126
lzp	0.259	0.545	0.436	0.100
ezp	0.188	0.328	0.266	0.058

- **Configuration:** 6_6_3_3_10

Runs: 17

Performance: Wide range in all metrics, highest lzp variance

Metric	Min	Max	Mean	Std
CP	0.050	0.327	0.237	0.059
Ps	0.357	0.786	0.546	0.114
lzp	0.050	0.521	0.382	0.108
ezp	0.007	0.382	0.257	0.076

- **Configuration:** Less Keywords

Runs: 21

Performance: Balanced performance across all metrics

Metric	Min	Max	Mean	Std
CP	0.173	0.327	0.233	0.048
Ps	0.357	0.786	0.582	0.118
lzp	0.293	0.591	0.431	0.094
ezp	0.202	0.382	0.265	0.056



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- **Configuration:** Standard

Runs: 24

Performance: Best overall average, highest lzp and ezp

Metric	Min	Max	Mean	Std
CP	0.165	0.318	0.247	0.048
Ps	0.357	0.786	0.589	0.132
lzp	0.282	0.577	0.443	0.099
ezp	0.189	0.381	0.279	0.056

4.3.3 Temporal Pattern

Metrics show variation over time with some configurations tested in batches. *Standard* and *Less Keyword* configurations show more consistent results. *Similarity Focus* configuration has the widest range of *Ps* scores (0.357-1.000).



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5 Visualization

5.1 *Development Benchmark*

The development benchmarks track performance improvements or regressions over time through two separate plots:

1. **Configurations Plot:** which shows individual test configuration performance over time
 - Individual configuration lines showing performance trends across benchmark runs
 - Score values annotated directly on the lines for precise reading
 - Y-axis range 70-90% for visibility
 - X-axis uses YYYY-mm-dd format with label “Date”
2. **Overall Average Plot:** which shows the combined performance across all configurations
 - Overall average line (black dashed line with diamond markers)
 - Score values annotated below the line
 - Y-axis range 70-90% for consistency

When multiple runs occurred on the same date, they are positioned in correct temporal order (left to right = earlier to later).



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

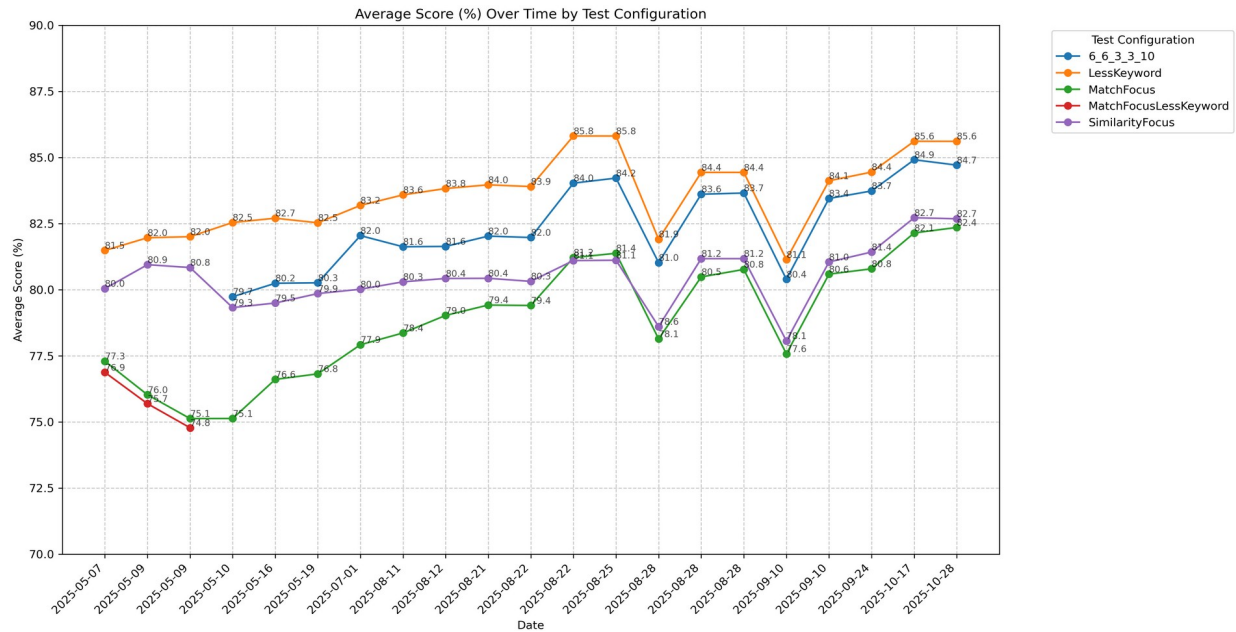


Figure 1: Figure 1: Average Score (%) Over Time by Test Configuration

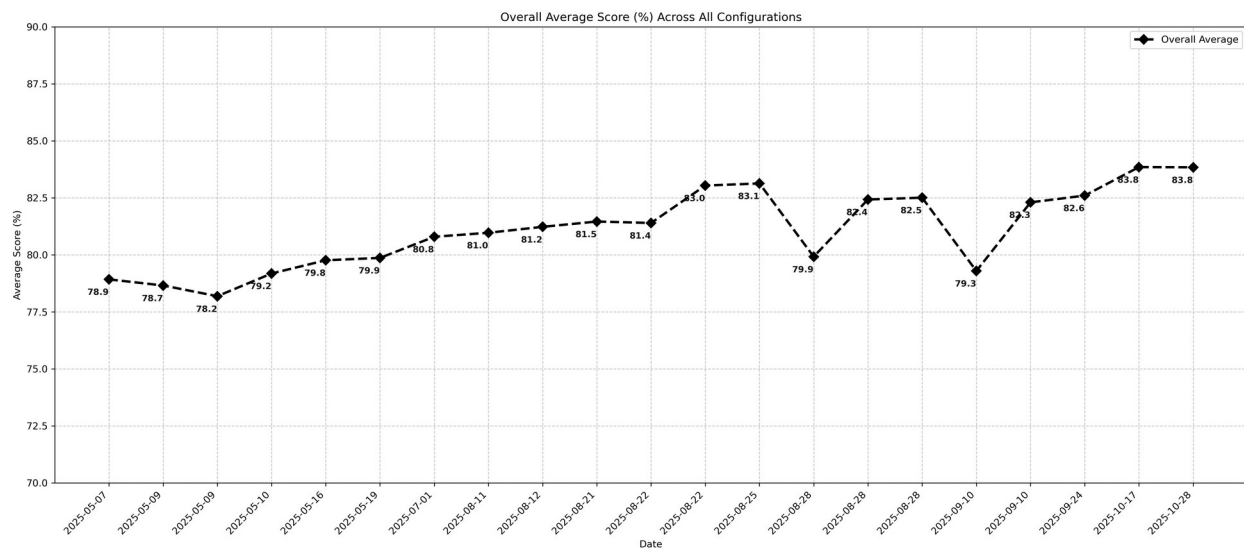


Figure 2: Figure 2: Overall Average Score (%) Across All Configurations



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2 Evaluation Benchmark Plots

5.2.1 Evaluation Run Metrics

In this paragraph, we show one plots for each metric, each showing the evolution over time of the run metric computed on all the test pair (c: comprehensive), only for expert test pairs (e), and only calculated for synthetic (s) test pair sets.

5.2.1.1 Mean Context Precision (MCP)

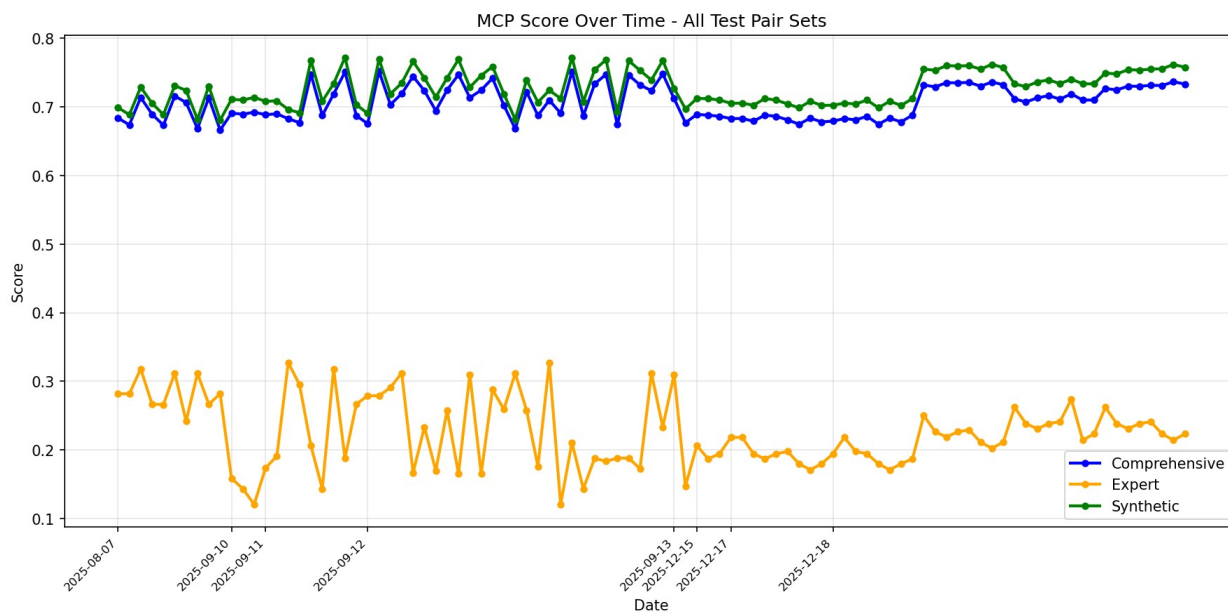


Figure 3: MCP Score Over Time



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.1.2 Mean Presence (Mps)

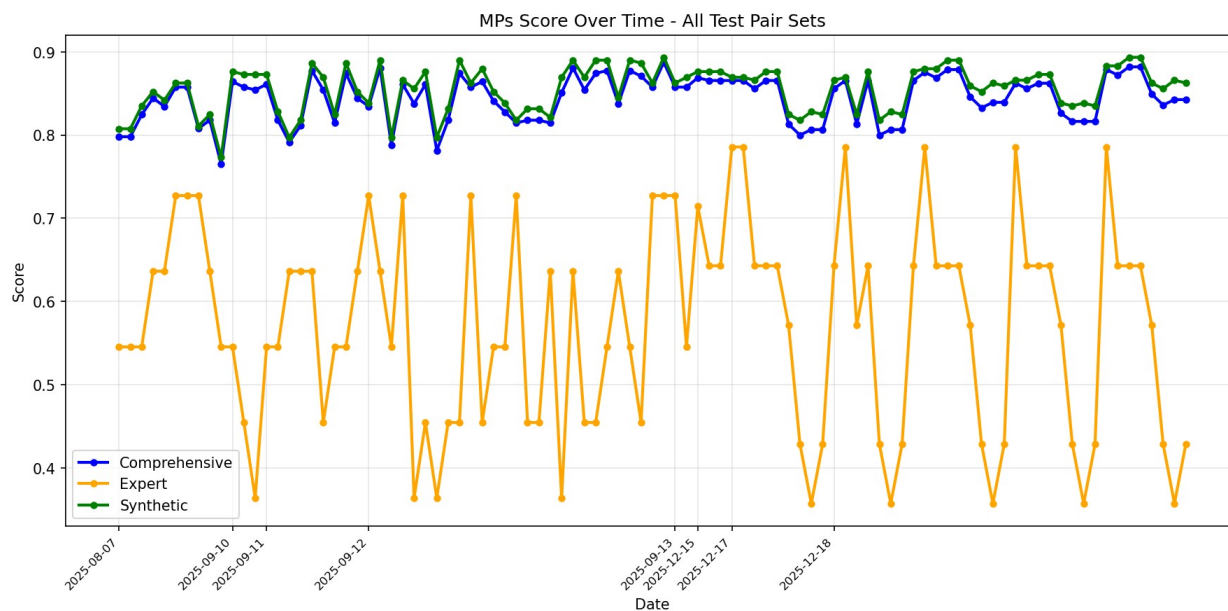


Figure 4: MPs Score Over Time



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.1.3 Mean Linear Zero Position (Mlzp)

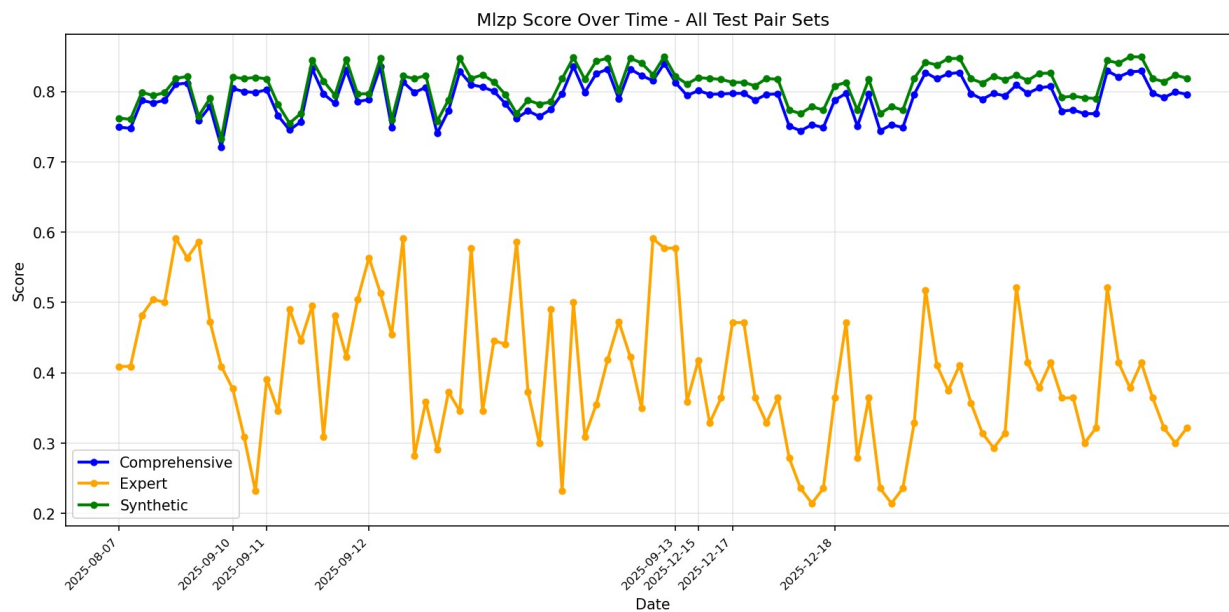


Figure 5: Mlzp Score Over Time



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.1.4 Mean Exponential Zero Position (Mezp)

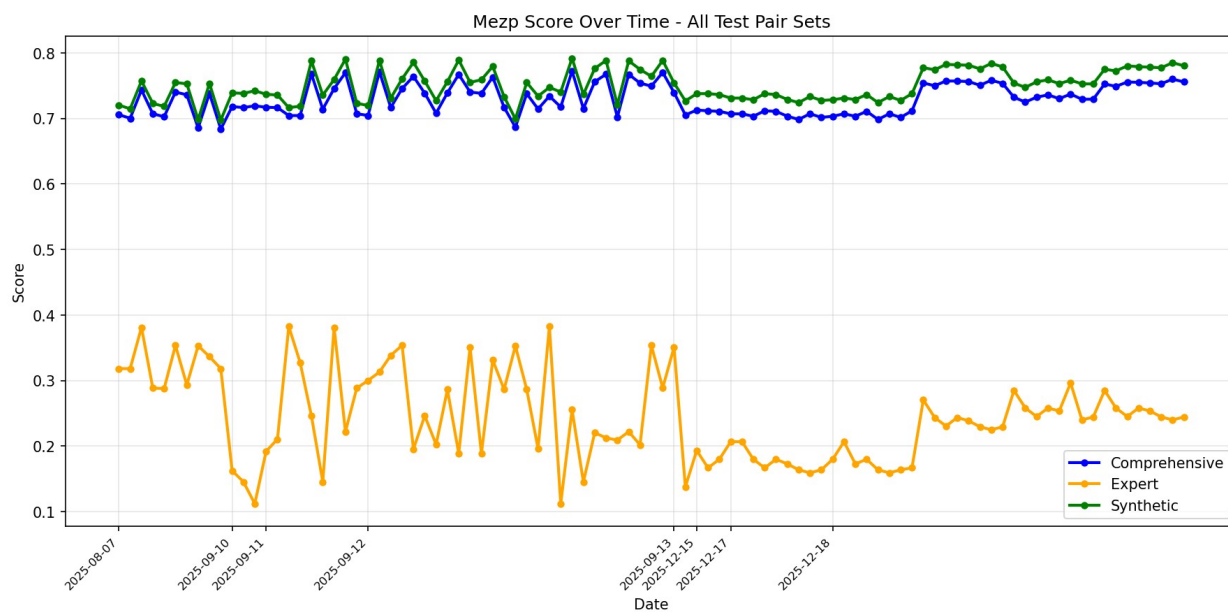


Figure 6: Mezp Score Over Time



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.2 Evaluation Test Pairs Metrics

5.2.2.1 Metrics Scores over time

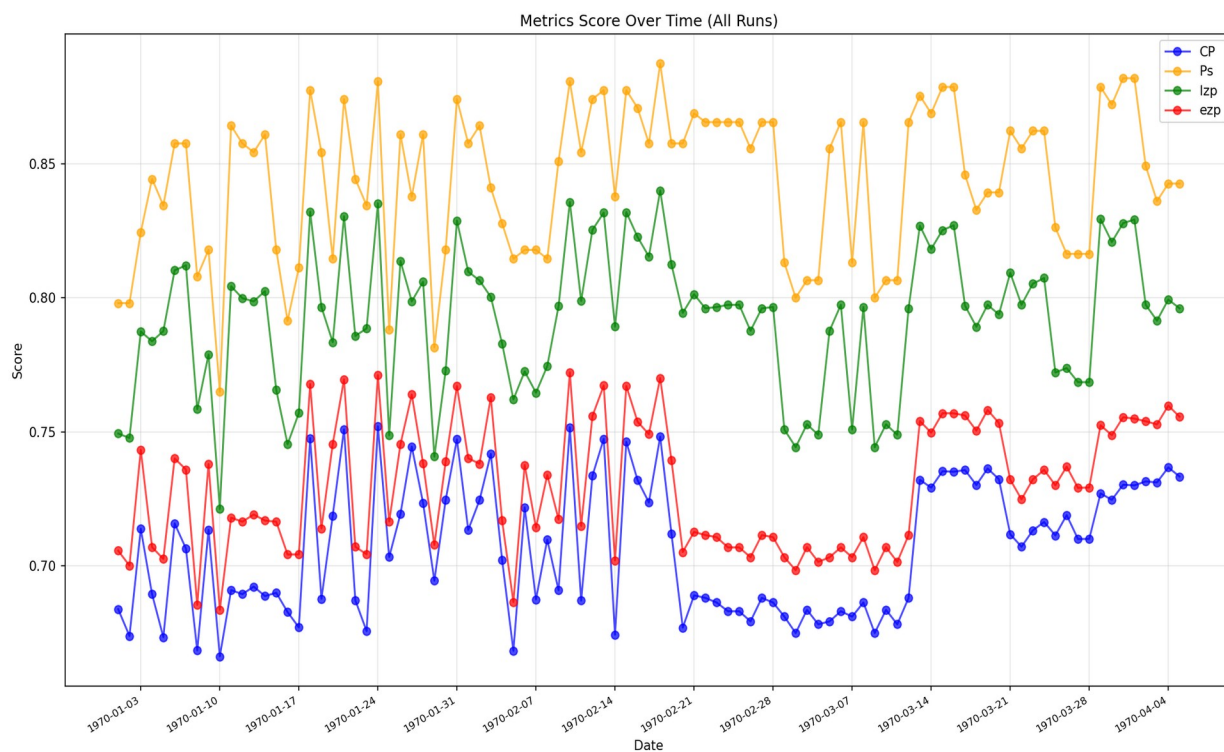


Figure 7: Metrics Score Over Time (All Runs)

This plot shows the evolution of CP, Ps, lzp, and ezp metrics across all runs, ordered chronologically. The x-axis displays dates in yyyy-mm-dd format, and the y-axis shows the metric scores. Each metric is represented by a distinct line with markers.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.2.2 Metrics Grouped by RFF Configuration

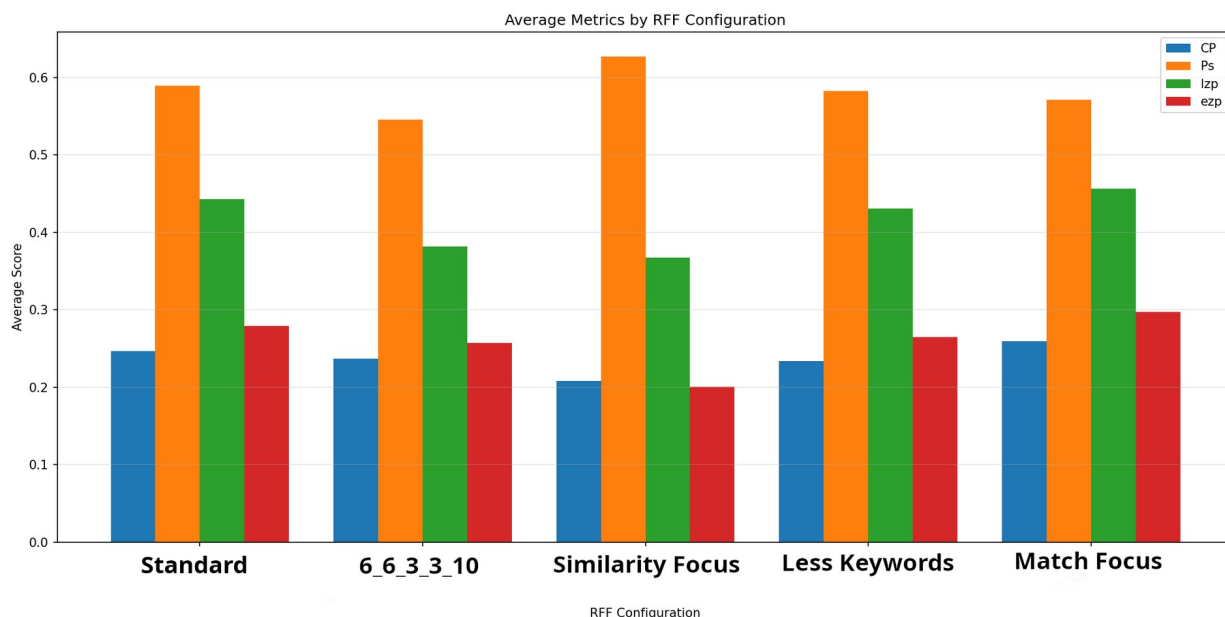


Figure 8: Average Metrics by RFF Configuration

This bar chart presents average scores for each metric (CP, Ps, lzp, ezp) grouped by RFF configuration. The x-axis labels identify each configuration, and the y-axis shows the average score. This visualization highlights which configurations perform best for each individual metric.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

5.2.2.3 Metrics Distribution

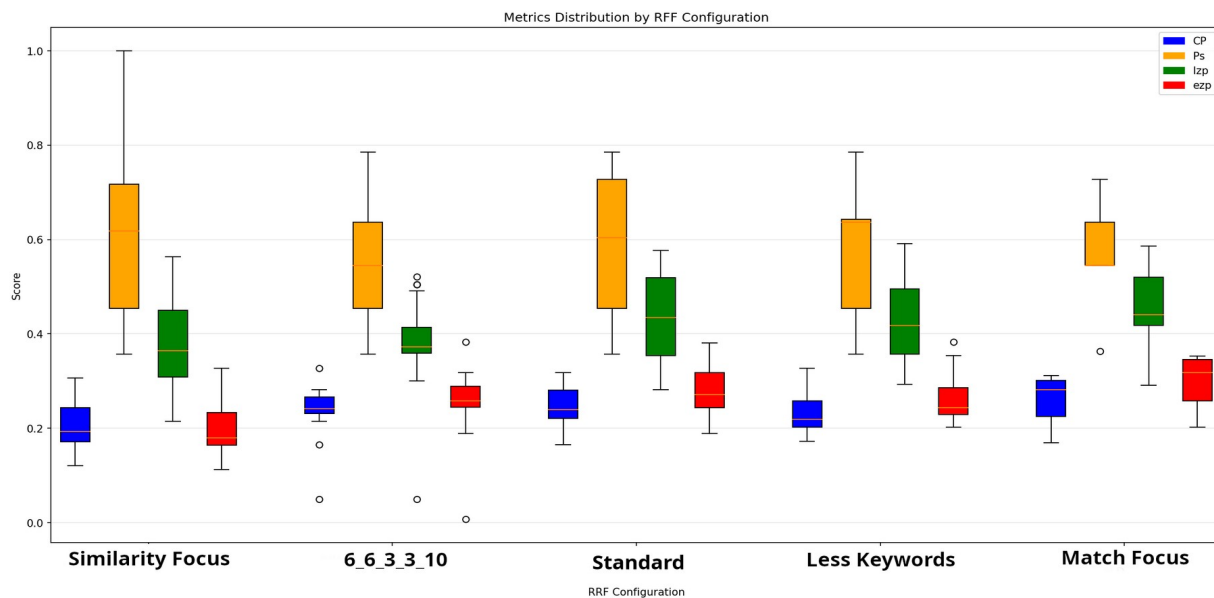


Figure 9: Metrics Distribution by RRF Configuration

This boxplot displays the distribution of each metric across all runs for each RRF configuration. It shows the median, quartiles, and potential outliers, providing insight into the consistency and spread of results for each configuration.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

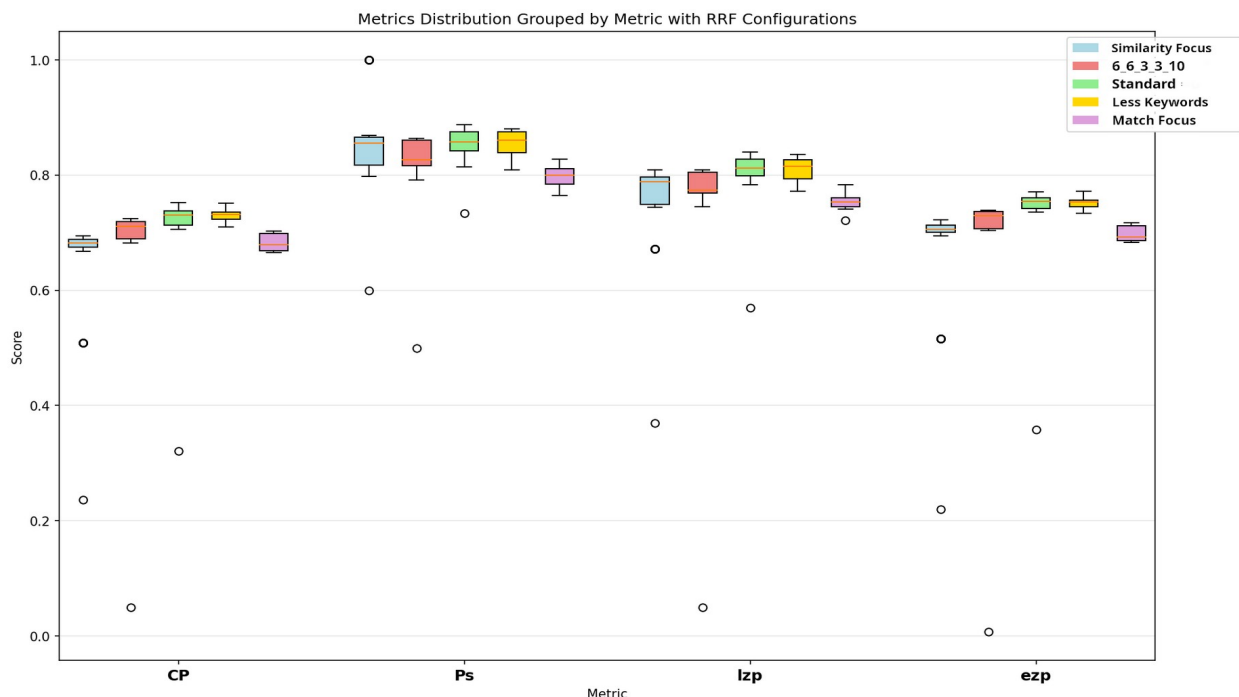


Figure 10: Metrics Distribution Grouped by Metric

Figure 10 contains a boxplot which organizes data by metric, with each metric (*CP*, *Ps*, *lzp*, *ezp*) having its own group of boxes representing the five RRF configurations. Each box shows the distribution of scores for that configuration on the given metric. The x-axis displays metric labels (*CP*, *Ps*, *lzp*, *ezp*) with configuration-specific boxes within each group. Different colors represent different configurations, as shown in the legend.

This plot facilitates the comparison on how different configurations perform on each individual metric and identify which configurations consistently produce higher scores for specific metrics.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

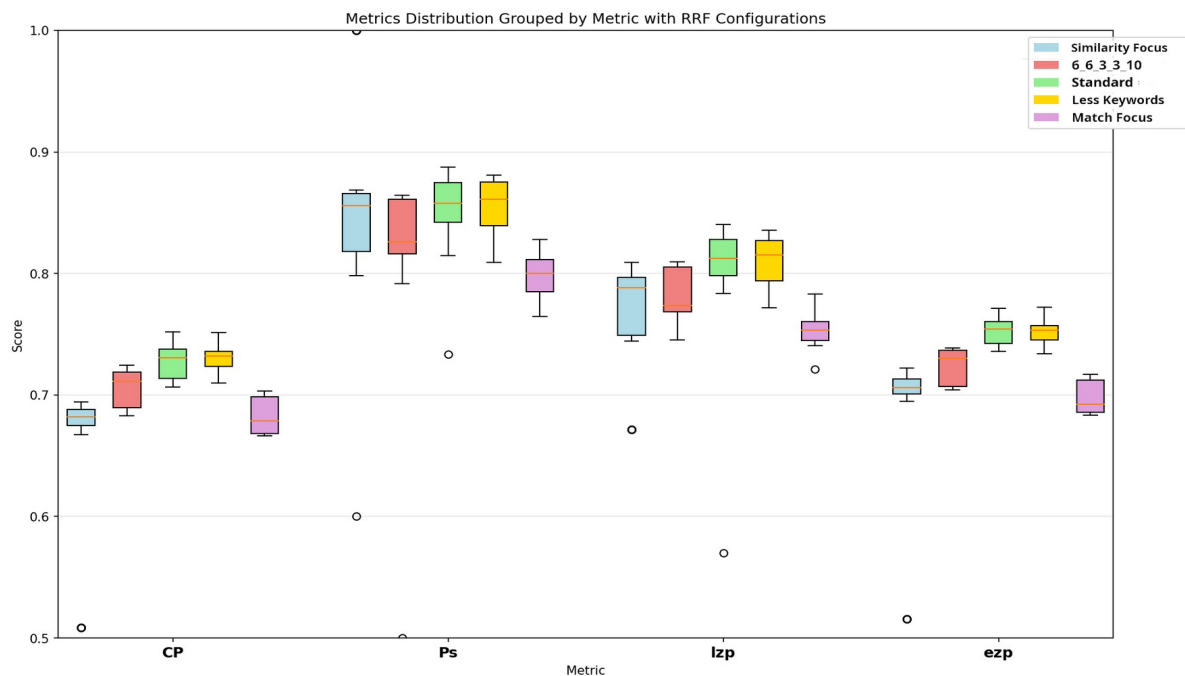


Figure 11: Metrics Distribution Grouped by Metric (Zoomed)

This zoomed version focuses on the upper range of scores (0.5 to 1.0), providing better visibility into the differences between configurations at higher performance levels.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

6 Strengths and Limitations

6.1 Strengths

1. **Comprehensive Testing Infrastructure:** The benchmarking system supports automated execution of hyperparameters configurations against multiple test pair sets (comprehensive, synthetic, expert). The development benchmarks additionally test predefined configurations with synthetic query datasets.
2. **Rich Metric Suite:** The system tracks 4 base metrics (*CP*, *Ps*, *lzp*, *ezp*) with additional 6 across multiple test pair sets, resulting in numerous distinct metric combinations stored individually and as run-level aggregations.
3. **Temporal Tracking:** Both development and evaluation benchmarks enable tracking of performance improvements or regressions over time. The development benchmarks use overall changes plots, while evaluation benchmarks track metrics across dates from August 2025 to present.
4. **Database-Driven:** All evaluation benchmark results are stored in a well-structured PostgreSQL database, enabling comprehensive analysis, historical tracking, and comparison across configurations and time periods.
5. **Statistical Rigor:** Uses large sample sizes (hundreds of queries across multiple datasets) and provides distribution visualizations (box plots, strip plots) to understand variability.
6. **Reproducibility:** The benchmark is fully automated with LLM caching, environment configuration (virtual environments, fixed seeds, deterministic temperature=0) ensuring consistent results across runs for the same configuration.
7. **Performance Metrics:** Tracks both accuracy (score) and efficiency (runtime), providing a balanced view of system performance.
8. **Component-Level Scoring:** Development benchmarks capture not just final ranking but also individual component scores (similarity, chunk, full/partial match, keyword), enabling detailed analysis of what contributes to good performance.
9. **Realistic Queries:** Uses synthetic but realistic query datasets that represent actual user search patterns, including facility-specific, instrument-specific, and general queries.
10. **Flexible Configuration:** The JSON-based hyperparameters files allow for easy experimentation with different RRF weightings, test pair sets and additional systems parameters.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

6.2 Limitations

1. **Synthetic Data:** While queries are designed to be realistic, they may not perfectly represent real user behavior or edge cases.
2. **LLM Dependency:** Results currently depend on GPT-4.1-mini's ability to correctly interpret and extract query parameters, which may introduce variability or biases.
3. **Fixed Decay Window:** The 20-rank decay window in development benchmarks is arbitrary and may not reflect actual user tolerance for result positioning.
4. **Database Dependency:** Results are tied to the current state of the OSCARS database, which changes over time as new entries are added.
5. **Configuration Space:** Only tests predefined configurations; there may be better configurations not yet explored.
6. **No User Feedback:** The benchmark doesn't incorporate actual user click-through or satisfaction data, relying solely on position-based scoring.
7. **Static Datasets:** The synthetic query datasets are static and may not evolve with changing user needs or new types of queries.
8. **Score Interpretation:** The linear decay model assumes uniform user dissatisfaction with lower positions, which may not reflect real user behavior.
9. **Incomplete Metric Coverage:** Out of 196 extended benchmark runs, only 117 have complete CP, Ps, lzp, ezp metrics, indicating some runs may have failed or been configured differently.
10. **Limited Expert Test Pairs:** Only 14 positive expert test pairs are available, which may not provide sufficient statistical power for comparing configurations.
11. **Metric Variance:** Some configurations (particularly Similarity Focus with low k values) show high variance in metrics like Ps, making it difficult to draw definitive conclusions.
12. **Missing Extended Presence Metrics:** The CP, Ps, lzp, ezp metrics are not directly computed for extended presence test pairs; only EP (Extended Presence) is available.
13. **Configuration Naming:** The hyperparameters files have descriptive names, but these names are not stored in the database, making it harder to correlate runs with their intended purposes without accessing the original JSON files.
14. **Database Schema Complexity:** The database schema with multiple tables and JSON fields can make queries complex, especially when trying to correlate test pair metrics with run configurations.
15. **Score Interpretation Differences:** The development benchmarks use a linear decay



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

model while Evaluation Benchmarks use RRF scoring, making direct comparisons between the two approaches challenging.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

7 Conclusion and Recommendations

7.1 Summary of Findings

The PaN-Finder benchmarking system represents a comprehensive framework for evaluating search performance across multiple dimensions. The different approaches - development benchmarks, evaluation benchmarks, both for test pairs and runs - provide a robust foundation for assessing system performance, identifying strengths, and pinpointing areas for improvement.

7.1.1 Performance Overview

- The development benchmarks show stable performance in the 75-90% range for most configurations, with facility and beamline-specific queries consistently scoring highest (92-100%)
- The evaluation benchmarks reveal that comprehensive and synthetic test pairs exhibit strong, consistent performance (mean scores 0.7064-0.8563), while expert test pairs present significantly greater challenges (mean scores 0.2255-0.5722)
- The extended test pair analysis confirms that Standard configuration provides the best overall performance (overall average 0.389) with the highest *lzp* and *ezp* scores, while Similarity Focus shows the lowest performance but highest variance
- Across all approaches, Presence (*Ps*) and Linear Zero Position (*lzp*) metrics consistently outperform Context Precision (*CP*) and Exponential Zero Position (*ezp*)

7.1.2 Temporal Stability

- Development benchmarks demonstrate relatively stable performance over the May-October 2025 period, with scores consistently in the 80-85% range for overall average
- Evaluation benchmarks span from August 2025 to present with regular testing, showing consistent patterns across configurations
- The Similarity Focus configuration, despite having the most runs (48), shows the widest range of performance, particularly in *Ps* metric scores

7.1.3 RRF Configuration Insights

- **Best Performers:** *Less Keyword*, *6_6_3_3_10*, and *Standard* configurations demonstrate the most consistent and highest performance



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

- **Challenging Configurations:** *Match Focus*, while conceptually appealing for precise matching, shows lower average performance in development benchmarks
- **Expert Test Gap:** All configurations struggle with expert test pairs, which show substantially lower performance and higher variability
- **Metric Consistency:** *Standard* and *Less Keyword* configurations show the most consistent results across all metrics

7.2 Recommendations

7.2.1 For Production Deployment

1. **Adopt Standard as Default:** The Standard configuration provides the best overall performance (highest overall average at 0.389) with the highest *lzp* and *ezp* scores. It should be used as the production default fallback.
2. **Continue Using 6_6_3_3_10:** The frontend default values (6_6_3_3_10) show strong, balanced performance and should continue to be used in production.
3. **Monitor Less Keyword Configuration:** The *Less Keyword* configuration performs well in development benchmarks and should be monitored as a potential alternative.

7.2.2 For Testing Infrastructure

1. **Expand Expert Test Coverage:** Increase the number of expert-curated test pairs from the current 14 to improve statistical significance of results and better reflect real-world usage.
2. **Include More Metrics for Extended Presence:** Compute *CP*, *Ps*, *lzp*, *ezp* metrics for extended presence test pairs to enable direct comparison with positive test pairs.
3. **Improve Configuration Tracking:** Store hyperparameters file names and descriptive configuration metadata in the database to facilitate correlation of runs with their intended purposes.
4. **Standardize Run Completion:** Ensure all runs complete successfully and compute all metrics to avoid gaps in the dataset (currently only 98 of 196 runs have complete metrics).
5. **Maintain Benchmarking Framework:** The benchmark framework is a valuable tool for ongoing development and should be maintained as part of the continuous integration process to catch any regressions from future updates.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

7.2.3 For Future Development

1. **Investigate Lower-Performing Entries:** Test pairs with lower scores (67-75% range in development benchmarks, expert test pairs in evaluation benchmarks) represent opportunities for improvement through better query understanding, ranking algorithms, or database indexing.
2. **Explore Additional Configurations:** Expand the configuration space, particularly configurations that might improve performance on lower-scoring datasets without degrading performance on high-scoring ones.
3. **Expand Query Diversity:** Incorporate more diverse query types and real user queries to better reflect actual usage patterns and ensure the system adapts to evolving user needs.
4. **Consider Alternative Scoring Models:** Evaluate whether the linear decay model and RRF scoring better reflects user satisfaction, and consider alternative approaches.
5. **Add User Feedback Mechanisms:** Consider incorporating actual user click-through or satisfaction data to complement position-based scoring and provide a more user-centric evaluation.



PaN-Finder Deliverable 3.b

Report on benchmarks and test pairs.

An OSCARS Project, Funded by the European Union

8 Acknowledgements

The authors acknowledge the assistance of Mistral Vibe (CLI agent, model: mistral-medium-3.5) in collecting the material, reviewing, analyzing, and providing stylistic recommendations for this document.

9 Funding

The PaN-Finder (Photon and Neutron federated knowledge finder) project is founded by the OSCARS project.

The authors acknowledge the OSCARS project, which has received funding from the European Commission's Horizon Europe Research and Innovation programme under grant agreement No. 101129751



**Funded by
the European Union**